Distributed Flash Sale Platform – Project Management

CS6650 Final Project

Shail Shah Vikas Neriyanuru Darshan Ravindra Konnur

April 2026

1 From proposal to final system

Original proposal (March 22). A three-service system (`waiting-room`, `flash-sale-api`, `order-worker`) on ECS Fargate, Redis for inventory, SQS for order hand-off, DynamoDB for final-state persistence. Three experiments: queue fairness, inventory correctness, admission-rate tuning.

Final system. The proposal's architecture landed intact. Infrastructure is Terraform in 8 modules (network, ALB, ECS, ECR, Redis, SQS, DynamoDB, logging). All three services are Go 1.21, each with its own Dockerfile and ECS task definition. A local-dev harness (Docker Redis + LocalStack for SQS/DynamoDB) was added late in the project to let us run experiments without burning ECS time.

The two changes between proposal and delivery:

1. Inventory strategies expanded from one to three. The proposal described atomic DECR as the default and optimistic locking as a failure-mode comparison; during Experiment 2 we added a third strategy (server-side Lua script) because we wanted a fair comparison between "safe but two round-trips" and "safe in one round-trip."
2. Admission-token enforcement was added in week 4. The waiting-room issued tokens from week 2, but the flash-sale-api did not check them. Without enforcement, Experiment 3 would have been measuring nothing – the rate limiter wasn't actually gating anything. We discovered this while running the Exp 1 sweep and found zero matches for `token` in the flash-sale-api source. A 40-line fix (header verify + env flag + integration test) closed the gap.

2 Work breakdown

Area	Owner	Status
Terraform (all 8 modules)	Shail	done
waiting-room service	Shail	done; two concurrency bugs fixed in week 4 (see section 4)
flash-sale-api service	Vikas	done; quantity fix + Exp 2 strategies + admission gate
order-worker service	Darshan	done
Experiment 1 harness + sweep	Vikas	done (Go harness; replaced the planned Locust version)
Experiment 2 harness + sweep	Vikas	done (3-strategy comparison)

Area	Owner	Status
Experiment 3 harness + sweep	Vikas	done (local parity stack; no ECS worker-count sweep)
Chart generation	Vikas	done (matplotlib)
Final report (5 pages)	all	done
Local-dev harness	Vikas	done (docker-compose + bootstrap script)
Video	all	scheduled

3 How we worked as a team

We operated like a small software team with clear ownership, a shared mainline, and explicit integration checkpoints rather than three people editing the same files ad hoc.

Ownership model. The architecture was split along service boundaries. Shail owned infrastructure and the waiting-room, Vikas owned the flash-sale-api, experiment harnesses, charts, and most of the final report plumbing, and Darshan owned the order-worker and the later Exp 2 automation / Locust work. This let each person move quickly inside a bounded area while still keeping the system design coherent.

Communication pattern. We coordinated through the shared GitHub repo, commit history, branch handoffs, and direct teammate check-ins before touching shared infrastructure or running expensive experiments. In practice this looked like lightweight standups: "what changed, what is blocked, what is safe to test, and what must stay stable on `main`." That mattered most in the final stretch, when one teammate was running Exp 2, another was testing Exp 3, and we needed to avoid invalidating each other's AWS state.

Scrum-like workflow. The proposal acted as the initial backlog. We then worked in short loops:

1. Build one vertical slice (infra, waiting-room, purchase path, worker).
2. Smoke-test that slice locally or on ECS.
3. Integrate it with the next service boundary.
4. Run one experiment or bug-reproduction pass.
5. Triage findings and only then expand scope.

That is why the repo history shows bursts of service work followed by integration / debugging commits rather than one giant "final project" drop.

GitHub / branch discipline. We kept `main` as the shared baseline and used feature or review branches when changes could destabilize active experiments. Late in the project, this became essential: rather than force everyone onto a moving `main`, we validated fixes on a review branch first, then fast-forwarded the shared branch only after the results, report, and harness files were all in place. The repository history itself is therefore part of the project-management artifact: it shows ownership, sequencing, stabilization work, and late bug triage.

Environment isolation. We also treated AWS accounts/environments like separate test environments in a company setting. Exp 2 and Exp 3 were run in separate places so one person's load test would not invalidate another's results. That decision reduced rework and made the final experiment numbers credible.

4 Timeline vs. plan

Week	Planned	Actual
1 (Mar 22-29)	Infra up; 3 services on Fargate E2E	Done; Terraform modules landed Mar 30
2 (Mar 30 - Apr 6)	Waiting room + strategies + harnesses	Waiting-room + flash-sale-api done; quantity fix Apr 3
3 (Apr 7-13)	Run all sweeps	Order-worker + smoke tests Apr 3-20
4 (Apr 14-19)	Report + deck	Bugs found + fixed Apr 20-21; all sweeps + report Apr 21

The week-3/4 compression was driven by late discovery that (a) the waiting-room INCR tiebreaker was silently broken, and (b) the admission token was never verified. Both were caught by the Experiment 1 data rather than by unit tests, which is itself a finding worth reporting in the lessons learned.

5 Problems encountered

1. Float64 precision destroyed Strategy B of Exp 1. The waiting-room's tiebreaker score was `ms + seq/1e9`. At 2026 timestamps, the sub-microsecond term was below float64 ULP at that magnitude, so the score saved to Redis was identical to `ms` alone. The code compiled, passed unit tests, and shipped; only the experimental sweep revealed that Strategy B's collision rate matched Strategy A's. Fix: drop the timestamp, use `INCR` alone as the score.
2. ZADD / ZRANK race in the waiting-room. `ZADD` and the follow-up `ZRANK` were separate round-trips. Concurrent inserts with lex-smaller member strings could displace an earlier caller's rank between the two calls, so two different users reported the same integer position at different wall-clock instants. Fix: move `INCR + ZADD + ZRANK` into a single server-side Lua script.
3. Admission-token gate not wired to flash-sale-api. See section 1, item
   2. This is the kind of bug that unit tests can't catch because each service's tests are correct in isolation.
4. Locust was not installed on the experiment host. We pivoted to Go harnesses that mirror the Locust load profile (zero spawn delay, per-request latency capture, CSV output). All three experiment harnesses are in `tests/exp{1,2,3}_*/` and reproducible with one command each.
5. LocalStack endpoint URL. The AWS SDK v2 respects `AWS_ENDPOINT_URL` for LocalStack; the order-worker needed no code change, only an env var. This saved ~2 days that would otherwise have gone into custom endpoint resolvers.

6 Final-stage process lessons

- Freeze feature work earlier and enter explicit stabilization mode. Once the experiments started producing real data, the project became more like release engineering than feature development. The most productive behavior in the last 24 hours was not "add more" but "keep the branch stable, rerun, verify, document."
- Make branch promotion a first-class step. The safe-review-branch pattern we used at the end should have existed from the start. It gave us a clean place to integrate fixes, results, PDFs, and harnesses before updating `main`.
- Treat experiments as production changes. A load harness or Terraform variable can change the behavior of the system just as much as a Go code patch. We learned to review experiment scripts and infra variables with the same seriousness as service code.

7 What we would do differently

- Add a contract test between services from day one. If flash-sale-api had an integration test that required a valid waiting-room token, the admission gate would have been wired in week 2 instead of week 4.
- Run experiments earlier, even with toy numbers. The Exp 1 bug would have surfaced in day three if we had sent 100 concurrent joins at the waiting-room and plotted collision rate. We waited until the full experiment was ready to run and missed two weeks of debugging runway.
- Pin the Go version across modules. `tests/flash-sale-api/go.mod` drifted to `go 1.25.5` while services use `go 1.21`; CI would have caught it, local development didn't.